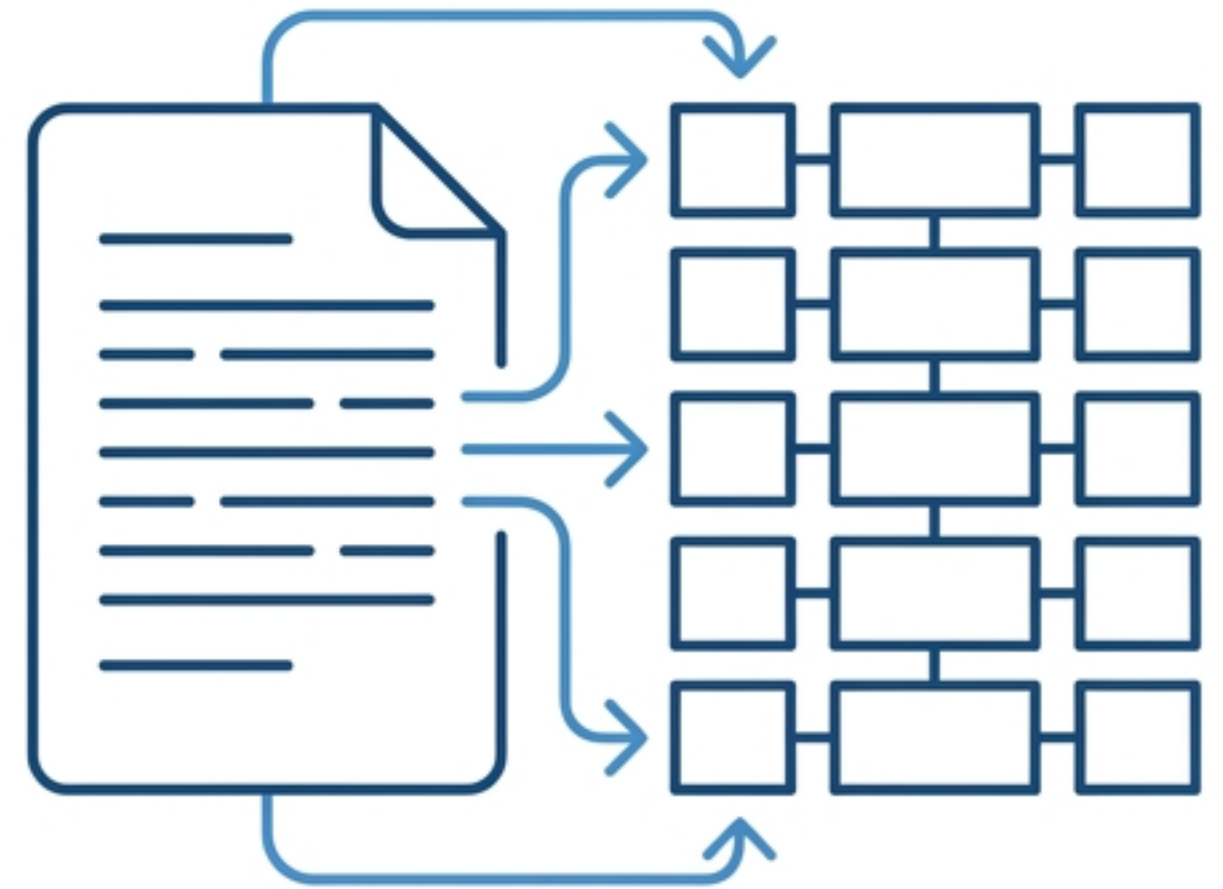


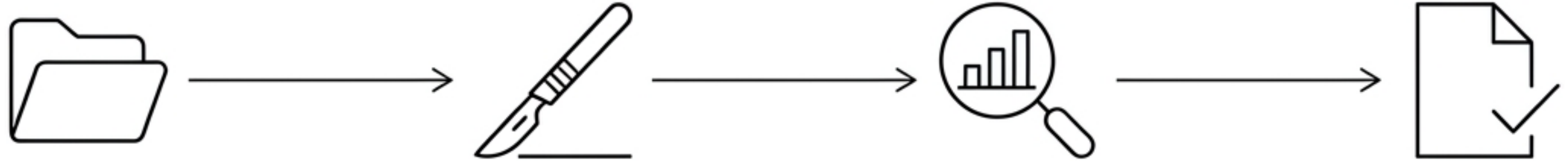
De Texto a Datos

Dominando Ficheros,
Cadenas y Listas en Python



- **Enfoque:** Extracción, Transformación y Carga (Pipeline ETL)
- **Orientado a:** Resolución de Casos Prácticos

El Ciclo de Vida de los Datos



1. Extraer

Leer el fichero de texto plano en memoria de forma segura.

2. Transformar

Limpiar caracteres ocultos y dividir las líneas en listas estructuradas.

3. Analizar

Recorrer las listas para buscar extremos (máximos/mínimos) o filtrar.

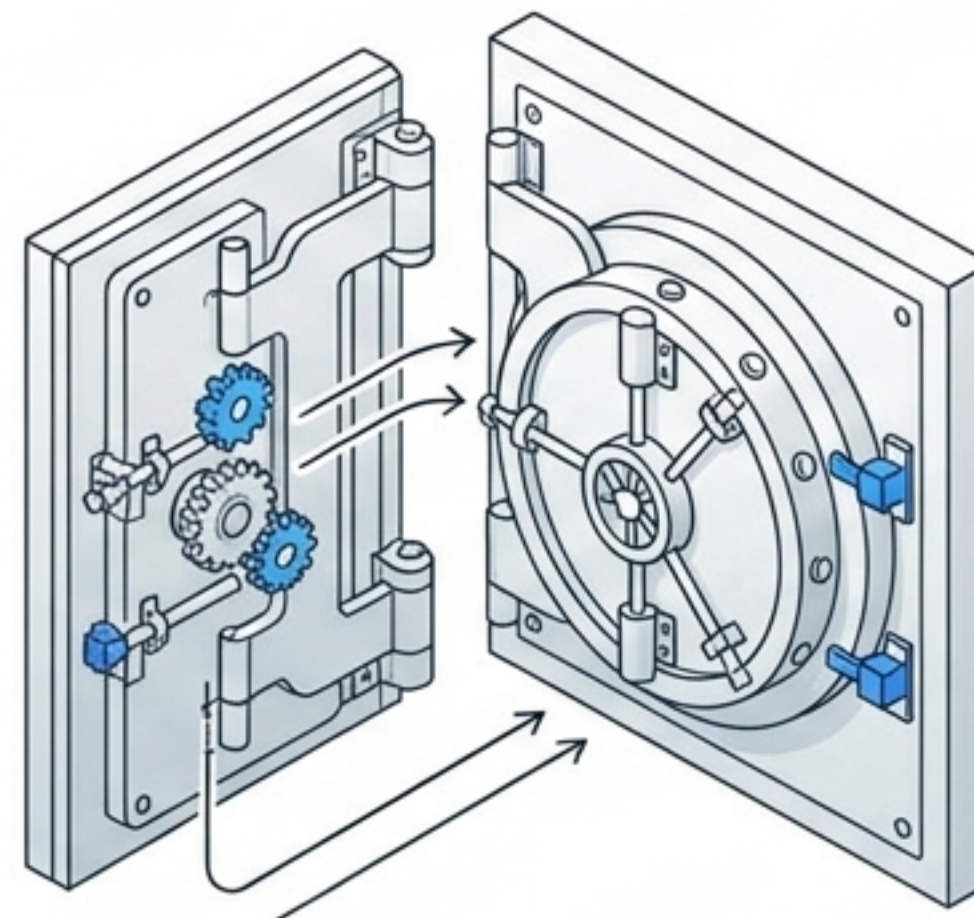
4. Cargar

Reensamblar la información y exportarla a nuevos ficheros.

Extracción: Apertura Segura Automática



```
archivo = open('prestamos.txt', 'r')  
lineas = archivo.readlines()  
archivo.close() # ¡Peligroso si hay un error antes!
```



```
with open('prestamos.txt', 'r') as archivo:  
    lineas = archivo.readlines()  
# Fuera del bloque, el archivo ya está cerrado.
```

El bloque 'with' actúa como un administrador de contexto. Garantiza que el fichero se cierre automáticamente, incluso si el programa falla. Elimina la necesidad de usar 'archivo.close()'.

El Kit de Limpieza de Cadenas

Método	Entrada → Salida	Caso de Uso Principal
<code>strip()</code>	<code>String → String</code>	Eliminar saltos de línea (<code>'\n'</code>) y espacios en blanco al inicio o final al leer un fichero.
<code>split(';')</code>	<code>String → Lista</code>	'Cortar' una línea de texto crudo en columnas separadas usando un delimitador.
<code>join()</code>	<code>Lista → String</code>	'Pegar' los elementos de una lista con un separador para escribirlos en un nuevo fichero.

Anatomía de una Transformación

1. Texto Crudo

`'2025-09-01; Rayuela; Julio Cortázar\n'`

2. Limpieza con strip()

`'2025-09-01; Rayuela; Julio Cortázar'\n'`

3. Corte con split(';')

`['2025-09-01', 'Rayuela', 'Julio Cortázar']`

Una línea de texto plano se convierte en un vector indexable de 3 elementos.

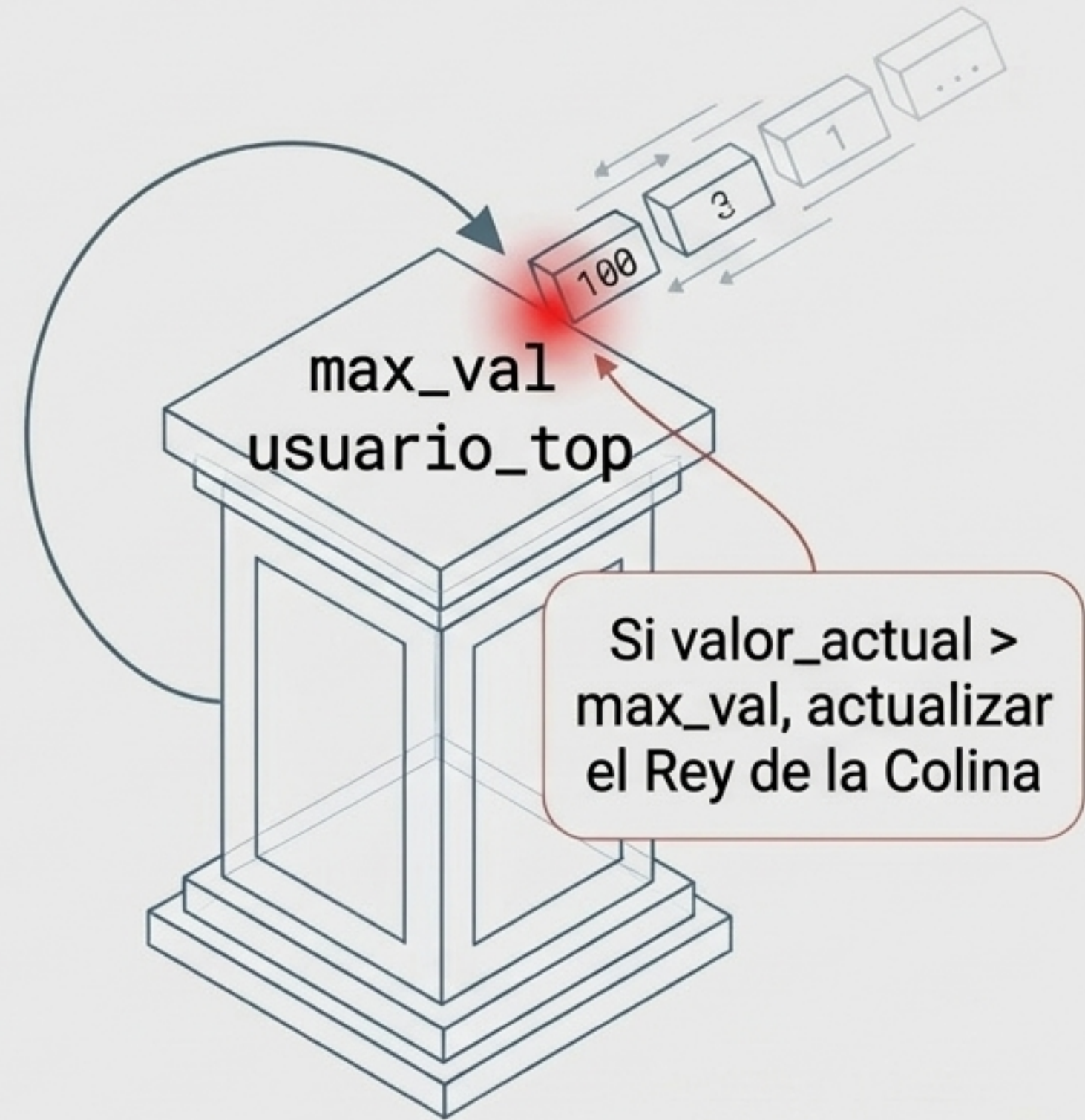
Estructurando: La Lista de Listas

	[0]	[1]	[2]	[3]	[4]
[0]	'2025-09-01'	'Rayuela'	'Cortázar'	'Ana'	'devuelto'
[1]	'2025-09-02'	'1984'	'Orwell'	'admin'	'pendiente'
[2]	'2025-09-03'	'El Extranjero'	'Camus'	'Carlos'	'retrasado'

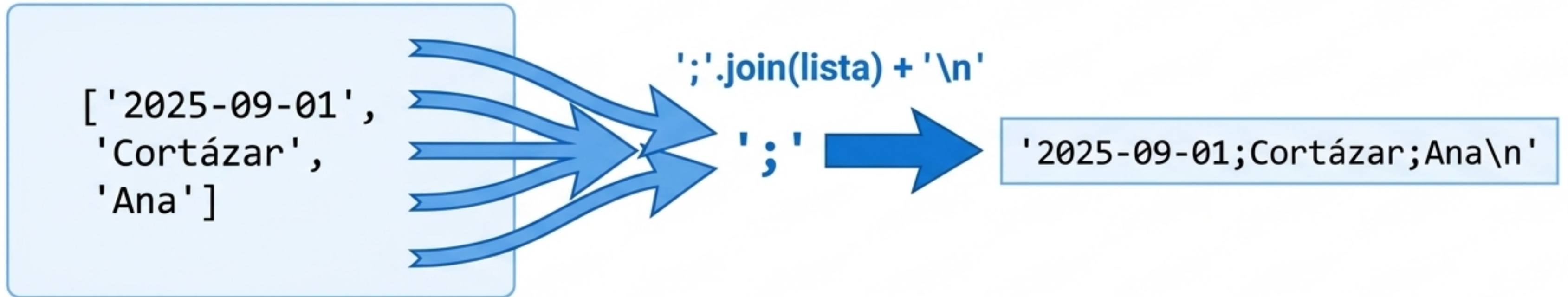
En memoria, el fichero completo se convierte en una matriz 2D. Para acceder al autor del segundo préstamo, usamos `'matriz[1][2]'`.
El bucle `'for linea in archivo:'` construye esta estructura fila por fila.

Análisis: Buscando Extremos

- Para encontrar los usuarios con más libros solicitados:
- 1. Crear un diccionario temporal para contar frecuencias.
- 2. Convertir el diccionario en una lista de listas: '[[usuario, cantidad], ...]'
- 3. **Recorrer la lista** comparando la posición "[1]" (cantidad) con una variable "max_val" inicializada en 0.
- 4. Si el valor **actual** supera al "max_val", se actualizan las variables de control.



Reensamblaje y Exportación



```
with open('informe.txt', 'w') as f:  
    for registro in datos_filtrados:  
        linea = ';'.join(registro)  
        f.write(linea + '\n')
```

El modo de apertura 'w' (write) crea o sobrescribe el fichero. El método `join()` es el opuesto exacto de `split()`.

Caso Práctico 1: Gestión de Préstamos (Filtros)

Requisitos del Examen:

- Ignorar los libros prestados al 'administrador'.
- Listar únicamente los libros no devueltos (estado distinto a 'devuelto').

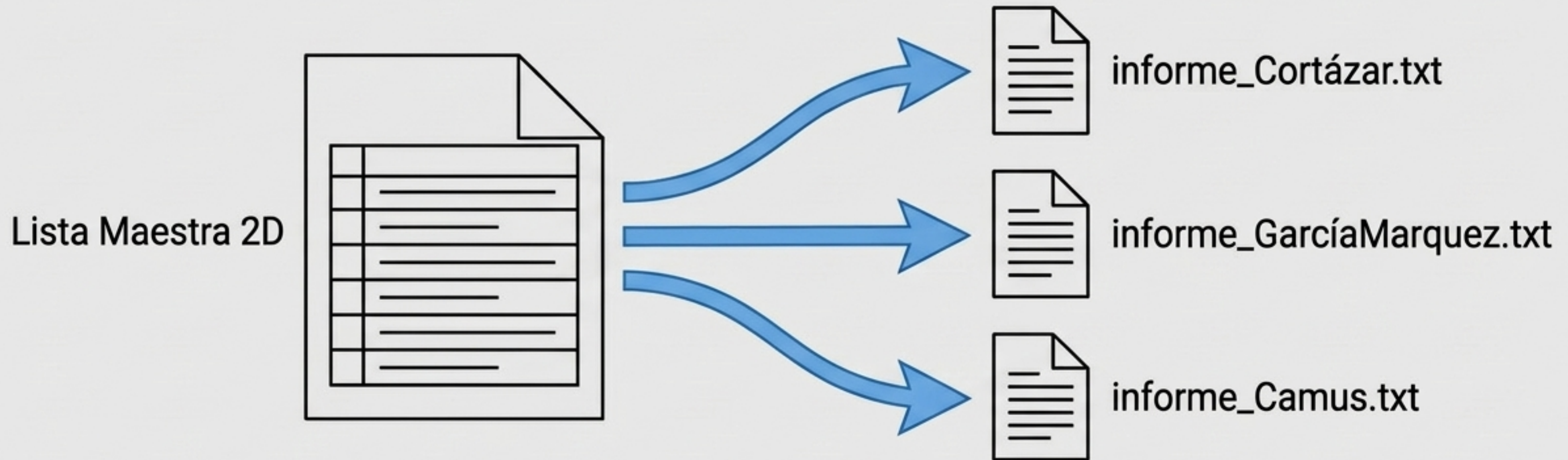
```
with open('prestamos.txt', 'r') as f:
    for linea in f:
        datos = linea.strip().split(';')
        usuario = datos[3].strip()
        estado = datos[4].strip()

        # Filtro de exclusión
        if usuario == 'administrador':
            continue

        # Filtro por estado
        if estado != 'devuelto':
            pendientes.append([usuario, estado])
```

Caso Práctico 1: Generación de Informes Dinámicos

Requisito: "Para cada autor crear un archivo con nombre: informe_[autor].txt"



1. Extraer una lista de autores únicos.
2. Iterar sobre esa lista de autores.
3. Por cada autor, abrir un fichero con formato f-string: **"with open(f'informe_{autor}.txt", "w") as f:"**
4. Buscar en la matriz original todos los registros que coincidan con ese autor y usar **"f.write()"** para guardarlos.

Caso Práctico 2: Validación de Cadenas

Validar '@'

Regla: Debe contener '@', pero no al inicio ni al final.

```
if '@' in mail and  
mail[0] != '@' and  
mail[-1] != '@':
```

Longitud Mínima

Regla: La longitud no puede ser menor a 7 caracteres.

```
if len(mail) >= 7:
```

Caracteres Especiales

Regla: Nombre sin números ni caracteres extraños.

Iterar sobre el string verificando cada caracter, o limpiar usando el método 'replace()'.

El Pipeline Completo: Cheat Sheet

1. Archivos

`with open('a.txt', 'r') as f:`
Apertura segura en modo lectura. Se cierra automáticamente.

`with open('b.txt', 'w') as f:`
Apertura segura en modo escritura.
Sobrescribe o crea.

2. Cadenas

`.strip()`: Limpieza de saltos de línea y espacios.

`.split(';')`: Cortar string crudo y convertirlo en lista.

`(';').join(lista)`: Pegar lista en string para exportar.

3. Listas y Matrices

`matriz[filas][cols]`: Acceso a elementos en estructura 2D.

`lista.append(registro)`: Agregar nuevos datos al vector.

`len(lista)`: Validar cantidad de elementos.

4. Lógica de Análisis

Uso de diccionarios temporales para conteo de frecuencias.

Bucles for para encontrar Máximos ('Rey de la Colina').

Filtros condicionales (if) para descartar registros.

En programación, la complejidad se resuelve dividiendo el problema: Extrae, Limpia, Analiza y Carga.